University of Exeter

Presented by :
Marcus Darrell, Ollie Pearce, Trystan Kiegher, Nehal Tumia and Aidan Taylor

Event Planner.

ECM1414 – Algorithms and Data Structures

University of Exeter



Sections

01 Introduction

02 Brute Force Approach

03 Dynamic Approach

04 Demonstration

05 Results

06 Discussion

07 Questions

University of Exeter

Introduction



Task: organize an events weekend

- Pre-existing list of activities with specific:
  - cost
  - duration
  - enjoyment value

University of Exeter

What's our problem?

PROBLEM 1

Limited resources

- Time
- Money

PROBLEM 2

Cannot choose same activity twice

- Combination of different activities

PROBLEM 3

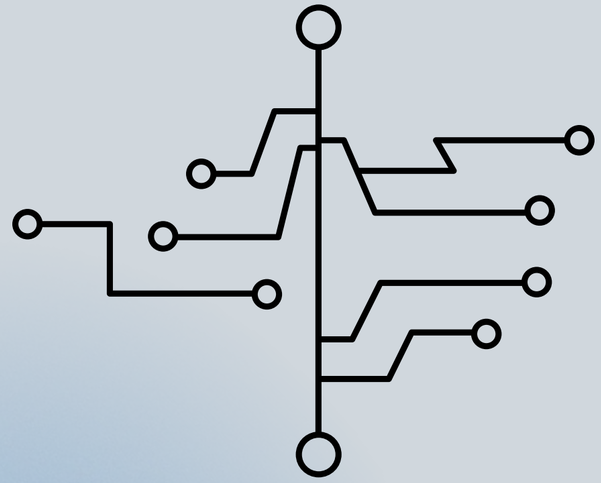
Maximise enjoyment value

- Must find optimal combination






University of Exeter



Our solutions



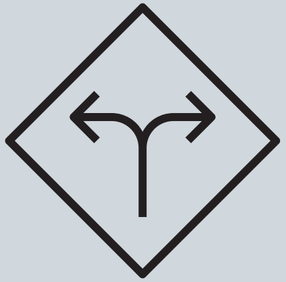
Solution 01

Brute Force Algorithm



Solution 02

Dynamic Algorithm



Solution 03

Greedy Heuristic

University of Exeter

What is Brute Force?



Iterates through **every possible combination** of events

- Order does not matter
  - Permutations formula : 2^n
- May be **huge** number of items depending on n (number of activities)

100 activities

1,267,650,600,228,229,401,496,703,205,376

items to iterate through!

Brute Force Algorithm

Combinations

Find number of all possible activity combinations

Permutation:
2^n

Binary

Represent each combination using **bit string**

For every digit in this bit string

- Append digit to **list**

Store values

Iterate through the list of digits in bit string

If digit is **1:**

- **Append** activity to list
- Store **enjoyment** value
- Store current **duration**

Constraints

Continue if :

- current duration < maximum duration

AND

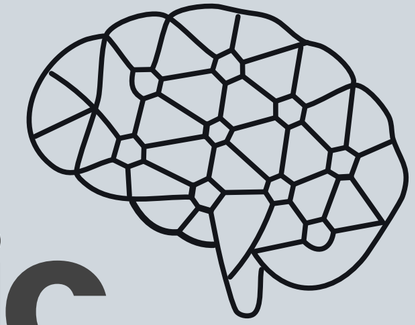
- current cost < budget

Comparison

If current enjoyment > best enjoyment:

- Update best combination
- Update best enjoyment value

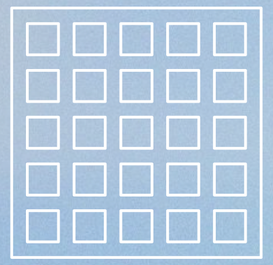
University of Exeter



What is a Dynamic Algorithm?

- Approach to recursive problems
- Results of subproblems are stored

We have used a matrix



Advantages compared to Brute Force:

- Avoid repeating calculations
- Skip unnecessary steps

=

Improve time complexity



Dynamic Approach Algorithm

Create Matrix --- **Include activity?** --- **Iterate through row** --- **Go backwards**

n x m

- **n = activities + 1**
- **m = max duration + 1**

Maximise enjoyment from this decision

- Do/ do not include activity

Repeat previous operation

- Until **end of row**
- Move to **next row**
- **Repeat** for every activity **until end** of matrix

Start at **bottom right element**
s = current duration

- Move up **1 row**
- Move **(s) steps left**

If value is equal

- **Do not add activity to best arrangement**

If value is not equal:

- **Add activity to best arrangement**

Results

activities	brute force time seconds	dynamic time seconds	greedy heuristics seconds
5	3.6500006914138796e-05	2.579999715089798e-05	1.2599999085068703e-05
10	0.0008608999997377396	4.679999500513077e-05	1.4600000344216823e-05
15	0.036752399995923044	6.089999526739121e-05	2.0099999383091926e-05
20	1.4319716999977827	8.799999952316284e-05	2.2299999371170997e-05
25	51.976652099996805	0.00011500000953674317	2.9500000178813933e-05
30	1938.6185229999944	0.00023349999636411667	3.159999940544367e-05
input_small	3.4499995410442355e-05	3.270000219345093e-05	1.3300000689923763e-05
input_small	3.460000455379486e-05	3.129999339580536e-05	1.2099999934434 89e-05
input_small	3.5899996757507325e-05	3.270000219345093e-05	1.2600000016391278e-05
average	3.499999890724818e-05	3.223333259423574e-05	1.2666666880249978e-05
input_medium	0.00398989999294281	5.950000137090683e-05	1.7500000074505806e-05
input_medium	0.0036188000068068504	5.2500002086162564e-05	1.5300000086426735e-05
input_medium	0.003907500006258488	5.5199995636940004e-05	1.4099999330937862e-05
average	0.0038387333353360497	5.573333303133646e-05	1.56333331639568e-05
input_large	57.22985329999775	0.00012550000846385957	2.4900000542402266e-05
input_large	56.414567999996244	0.00013439999520778657	2.3300000466406347e-05
input_large	54.64805209999532	0.00011760000139474868	2.1999999880790 71e-05
average	56.097491133329775	0.00012583333350221316	2.3400000296533105e-05

Discussion

Out of the three algorithms:

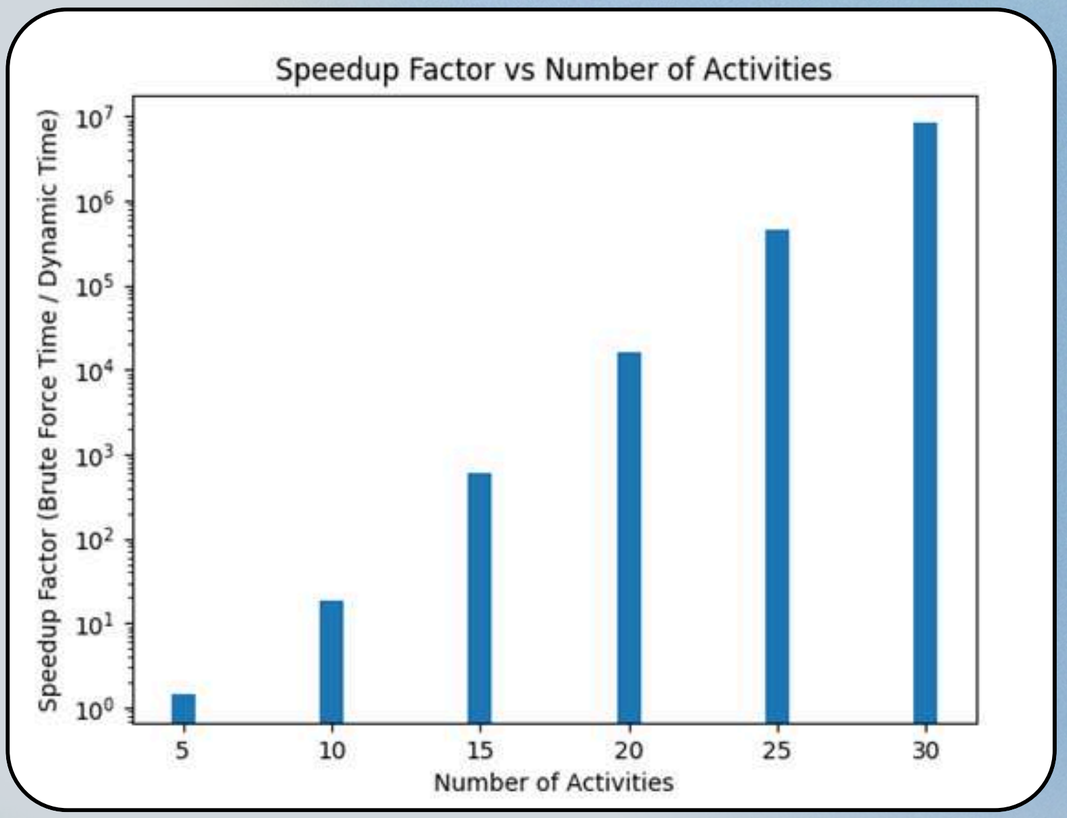
- **Greedy Heuristic** has the **shortest** execution time

Difference in execution time

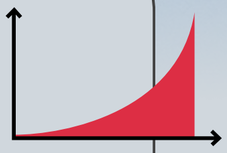


- Brute force **x445,807** Dynamic
- Brute force **x2,397,328** Heuristic
- Dynamic **x5.37** Heuristic

- Dynamic + Heuristic most similar



Difference in execution time grows exponentially



y - ‘Speedup Factor’
(Brute Force execution time / Dynamic Time)
x - number of activities

University of Exeter

Presented by :
Marcus Darrell, Ollie Pearce, Trystan Kiegher, Nehal Tumia and Aidan Taylor

Thank you for you time and attention!

Questions?